SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code	**CSA4315**	**Course Title**	**INTERNET PROGRAMMING**
CO Assessed	**CO1**	**Assessment**	**Assessment Tool 1: Application Based Questions**
Weightage	**40% of CIA**	**Total Marks**	**100**
CO1 Statement	*Explain the fundamentals of web architecture, client-server communication, Internet protocols, HTTP request and response messages, and configure LAMP/WAMP environments for web application development and deployment.* **(BL3)**		
Student Name	**BANDLAPALLI BHANUTEJA REDDY**	**Reg. No.**	**192325016**
Section/Batch	**Slot D / AI&ML**	**Date**	**20-10-2026**

Code of Conduct:

I , BANDLAPALLI BHANUTEJA REDDY , certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



How a Login Form Submission Works

5. A student submits a login form on a website. Explain how the browser constructs an HTTP request and how the server processes the submitted credentials.

Introduction

Almost every website has a login page, and most of us type a username and password into it without thinking about what happens next. In reality a fair amount of work takes place between the moment the Login button is clicked and the moment the next page appears. The browser has to turn the form into a correctly formatted message, find the server on the internet and send the message to it. The server then has to read the message, check the credentials against what it has stored, and send back an answer that the browser can understand.

This answer explains that journey using a simple example. A student opens the login page of a college portal at portal.example.edu, types an email address and a password, and presses Login. The explanation is divided into two parts. The first part covers how the browser constructs the HTTP request. The second part covers how the server processes the submitted credentials and replies. The server is assumed to run Apache, PHP and MySQL, which is the usual LAMP setup used in this course.

1. Client-Server Communication and HTTP

The web works on the **client-server model**. The client is the program that asks for something, which here is the student's browser. The server is the machine and software that holds the website and answers the requests. The two talk using **HTTP** (HyperText Transfer Protocol), an application layer protocol that runs on top of TCP. When the address begins with https, the same HTTP messages are simply carried inside an encrypted TLS connection.

HTTP follows a **request and response** pattern: the client always sends a request first and the server replies with one response. HTTP is also **stateless**, so the server does not remember anything about earlier requests by itself. This matters for logins, because once the student is logged in the server must still recognise the same student on the next click. That problem is solved with sessions and cookies, described in Section 5.

Every HTTP message has the same structure. A request starts with a request line (the method, the path and the protocol version), followed by header lines giving extra information, then an empty line, and finally an optional body that carries data. A response has the same shape but begins with a status line containing a status code such as 200 or 302.

2. Part A: How the Browser Constructs the HTTP Request

The login form on the page has an action pointing to /login.php, uses the POST method, and contains an email field, a password field and a hidden field holding a security token. Figure 1 shows the main steps the browser goes through after the Login button is pressed.



Figure 1: Steps taken by the browser after the student clicks Login

2.1 Collecting and checking the input

While the student types, the browser only keeps the characters inside the input boxes and nothing is sent to the network. When Login is clicked, the browser first runs its built-in **client-side validation**. It checks, for example, that no required field is empty, that the email looks like a proper email address, and that the password meets the minimum length. If a check fails, the browser shows a message beside the field and stops, so no request is made. JavaScript on the page can add further checks. These checks are only a convenience for the user, because anyone can bypass them, so the server has to repeat all of them.

2.2 Finding the server and opening a connection

The browser combines the form action with the current page address to get the full URL, https://portal.example.edu/login.php, and splits it into the scheme (https), the host name, the port (443 by default) and the path. Computers route traffic by IP address and not by name, so the browser looks up the host name using **DNS**. It checks its own cache and the operating system cache first and then asks a DNS resolver, which returns an IP address.

Next the browser opens a **TCP connection** to port 443 using the three-way handshake (SYN, SYN-ACK, ACK). Because the address uses https, a **TLS handshake** follows. The server presents its certificate, the browser checks that it is valid for this domain, and both sides agree on encryption keys. From then on everything sent on the connection, including the password, is encrypted. Without HTTPS the password would travel as plain text and anyone on the same network could read it. If a connection is already open, modern browsers reuse it (keep-alive) and skip this step.

2.3 Building the request message

With the connection ready, the browser builds the HTTP request. The first decision is the **method**. The form specifies POST, which is the right choice for a login. A GET request would place the values in the URL, where they could be saved in browser history, written to server logs and leaked through the Referer header. With POST the values are placed in the message body instead.

The browser then creates the **body**. Each named field becomes a key and value pair, for example username=student01@example.edu. By default the pairs are written in the **application/x-www-form-urlencoded** format: they are joined with the & symbol and special characters are percent-encoded so they cannot be confused with the separators (the @ sign becomes %40 and a space becomes %20). So the body of our request contains the username, the password and the hidden token as three encoded pairs.

Finally the browser adds the **headers**. The important ones are described below, and the complete structure of the message is shown in Figure 2.

- **Host** tells the server which website is wanted, because one server can host many sites.
- **Content-Type** describes the body format, here application/x-www-form-urlencoded, so the server knows how to read it.
- **Content-Length** gives the size of the body in bytes, so the server knows where the message ends.
- **User-Agent, Accept and Accept-Language** describe the browser and the kinds of response it can handle.
- **Origin and Referer** say which page the request came from, which helps the server reject forms posted from other websites.
- **Cookie** returns any cookie stored earlier, such as the session id issued when the login page was first loaded.

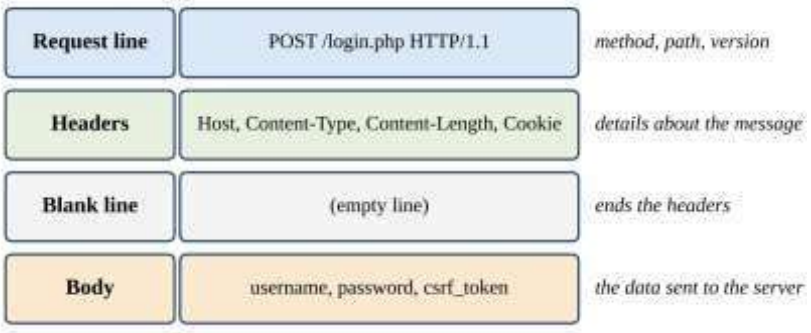


Figure 2: Structure of the HTTP POST request sent by the browser

The browser writes this message into the encrypted connection and waits for the reply. If the page uses JavaScript (fetch or AJAX) the idea is the same, except that the script builds the body itself, often as JSON with the Content-Type application/json, and handles the response without reloading the page.

3. Part B: How the Server Processes the Credentials

3.1 Receiving and parsing the request

Apache listens on port 443. It accepts the connection, completes the TLS handshake, decrypts the data and reads the request line and the headers. The Host header selects the correct virtual host and the Content-Length header tells Apache how many bytes of body to read. The path /login.php is then mapped to a file in the document root, and because the file ends in .php, Apache passes the request to the PHP interpreter. PHP reads the body according to the Content-Type and fills the $_POST array with the submitted fields, and it fills $_COOKIE from the Cookie header.

3.2 Server-side validation

The script now checks the input again, because the server can never trust data that arrives from outside. It confirms that the method is POST, that both fields exist and are not empty, that the email has a valid format and that the lengths are reasonable. It also compares the hidden token with the one stored in the user's session. If the token is missing or wrong, the request is rejected as a possible **CSRF** (cross-site request forgery) attempt.

3.3 Looking up the user

The script connects to the MySQL database and searches the users table for the submitted email. This must be done with a **prepared statement**, where the SQL command and the user input are sent to the database separately. If the input were simply joined into the SQL text, an attacker could type something like ' OR '1'='1 and log in without knowing any password. This attack is called **SQL injection**, and prepared statements prevent it.

3.4 Verifying the password

A properly designed system never stores passwords as readable text. It stores a **hash**, which is the result of a one-way function. Algorithms such as bcrypt and Argon2 are deliberately slow and add a random **salt** to every password, so two users with the same password end up with different hashes and precomputed tables become useless. The server cannot decrypt a hash. Instead it hashes the password that was just typed, using the same salt and settings, and compares the result with the stored value. In PHP this is done with password_verify(), which also compares in constant time so that timing does not leak information.

If the email is not found, a good implementation still performs a dummy hash check and gives the **same generic message** ("Invalid email or password") in both cases. This stops an attacker from learning which emails are registered.

3.5 Creating the session

If the hashes match, the student is authenticated. Since HTTP is stateless, the server creates a **session**, which is a record stored on the server that holds the user id and is identified by a long random **session id**. The script first issues a fresh id (session_regenerate_id) to prevent session fixation. The id is then sent to the browser in a **Set-Cookie** header. The cookie should carry the flags HttpOnly (JavaScript cannot read it), Secure (sent only over HTTPS) and SameSite (not attached to requests started from other sites). From now on the browser sends this cookie with every request, and the server uses it to find the session and recognise the student.

3.6 Sending the response

For a successful login the server usually does not return the dashboard directly. It sends a **302 Found** response with a Location header pointing to the dashboard and the Set-Cookie header described above. This is the Post/Redirect/Get pattern, and it means that refreshing the page does not submit the form a second time. If the login fails, the server sends the login page again with the error message, normally with status 200 (or 401 for an API), and no session is created. Figure 3 summarises the server-side processing.

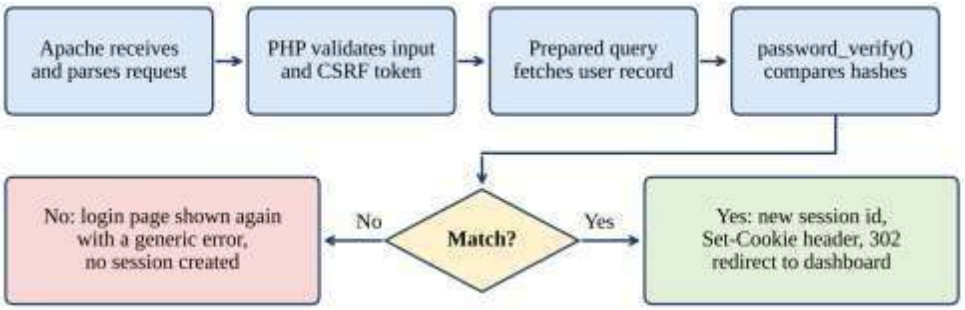


Figure 3: Server-side processing of the submitted credentials

4. What the Browser Does With the Response

The browser reads the 302 response, stores the session cookie, and follows the Location header by sending a new GET request for the dashboard. This second request carries the cookie, so the server can look up the session, see that a user id is stored, and build the personalised dashboard. It replies with 200 OK and the HTML, which the browser then displays. To the student all of this feels like one click. Figure 4 shows the complete exchange between the browser, Apache, PHP

and the database.

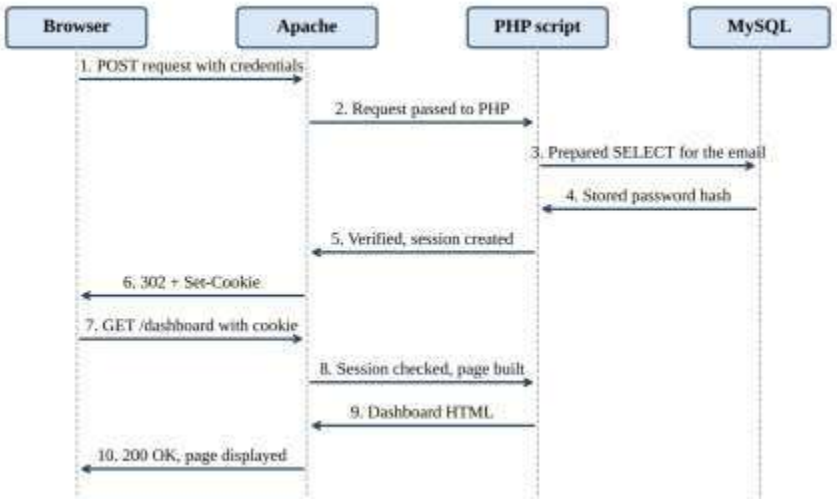


Figure 4: Message flow for a successful login

5. What Happens When the Login Fails

Not every login attempt succeeds, and the server has to handle failures carefully. A failure can happen at several points: a field may be missing, the CSRF token may be wrong, the email may not exist, or the password may not match the stored hash. In every case the server stops without creating a session. It then sends the login page back to the browser with a short message, normally with status 200, so that the student can try again. The message is deliberately the same for a wrong email and a wrong password, so that nobody can use the page to check which accounts exist.

Servers also keep track of repeated failures. After a number of wrong attempts from the same account or the same IP address, the server can slow down its responses, show a CAPTCHA, lock the account for a short time, or return status 429 Too Many Requests. Failed attempts are written to a log file, which allows administrators to notice an attack in progress. None of this is visible to a normal student, but it is an important part of how a real login system protects its users.

6. Common Status Codes and Security Measures

Several status codes appear in a login flow. **200 OK** is returned when a page loads normally or when the login page is shown again after a wrong password. **302 Found** redirects the browser after a successful login. **400 Bad Request** means the request was malformed, **401 Unauthorized** means authentication failed, and **403 Forbidden** means the request was refused, for example because the CSRF token was wrong. **429 Too Many Requests** appears when rate limiting stops repeated failed attempts, and **500 Internal Server Error** means the script itself failed.

Because a login form is the entrance to the whole application, it needs several layers of protection:

- Use HTTPS everywhere and send credentials in a POST body, never in the URL.
- Use prepared statements to stop SQL injection and escape any user data shown back on a page to stop XSS.
- Store only salted hashes made with bcrypt or Argon2, never plain text or fast hashes such as MD5.
- Limit login attempts, add delays or a CAPTCHA, and offer two-factor authentication to resist brute force attacks.
- Use CSRF tokens, SameSite cookies, a new session id after login, and session expiry to protect sessions.
- Show one generic error message so that attackers cannot discover which accounts exist.

7. Conclusion

Submitting a login form looks simple, but it shows the whole client-server model at work. The browser validates the fields, finds the server through DNS, opens a TCP and TLS connection, and builds an HTTP POST request whose headers describe the message and whose body carries the URL-encoded credentials. The server receives the request through Apache, hands it to PHP, validates the data again, finds the user with a prepared statement and checks the password against a stored hash. On success it creates a session, returns the session id in a Set-Cookie header and redirects the browser, and later requests are recognised through that cookie. Understanding this sequence explains why POST and HTTPS are used for passwords, why servers never store real passwords, and why client-side checks can never be the only defence.

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Web architecture and HTTP basics	15	Clear, correct account of client-server model, statelessness and message format	Mostly correct, minor gaps in HTTP basics	Partly correct, some points missing	Mostly incorrect or very brief	
Browser builds the HTTP request	25	Full flow: validation, DNS, TCP/TLS, POST method, encoding and headers explained	Main steps covered, few details missing	Only some steps, limited explanation	Request constructio n unclear	
Server processes the credentials	25	Parsing, validation, prepared query, hash check, session and response all explained	Most server steps explained correctly	Few steps, little detail on hashing or sessions	Server side barely explained	
Sessions, cookies and security	20	Session id, Set-Cookie flags, CSRF, SQL injection and HTTPS explained with reasons	Key security points listed with some reasons	Security mentioned without explanation	Little or no security discussion	
Presenta tion and diagram s	15	Well organised, clear language, relevant figures with captions	Organised, few figures or small errors	Average structure, unclear in places	Poorly organised	